EE 446 TinyML — Knowledge Distillation with Pruning and Quantization

Student TODO Version: Compression of a DNN Using the UCI Human Activity Recognition Dataset

In this version, key parts of the notebook have been left for you to complete.
Follow the instructions in each code cell and fill in the missing sections marked with `#<---Enter your code here--->#`.

1. Environment Setup

This notebook assumes you are running it with the **`Python (tinyml-arduino)`** kernel.

Expected environment:

- TensorFlow 2.14.1
- TensorFlow Model Optimization 0.8.0
- NumPy, Pandas, Matplotlib, Scikit-learn
- No in-notebook package reinstallation is required

Use **Kernel** → **Change Kernel** → **`Python (tinyml-arduino)`** if needed.

In [1]:
```
import os
import math
import zipfile
import random
import urllib.request
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

from tensorflow_model_optimization.sparsity.keras import (
```

```
    prune_low_magnitude,
    PolynomialDecay,
    UpdatePruningStep,
    strip_pruning
)

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow version:", tf.__version__)
```

```
2026-04-23 21:43:18.276778: I tensorflow/core/util/port.cc:111] oneDNN custom operat
ions are on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the enviro
nment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-04-23 21:43:18.317232: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find
cuda drivers on your machine, GPU will not be used.
2026-04-23 21:43:18.576488: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.
cc:9342] Unable to register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
2026-04-23 21:43:18.576589: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.
cc:609] Unable to register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
2026-04-23 21:43:18.576798: E tensorflow/compiler/xla/stream_executor/cuda/cuda_bla
s.cc:1518] Unable to register cuBLAS factory: Attempting to register factory for plu
gin cuBLAS when one has already been registered
2026-04-23 21:43:18.751362: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find
cuda drivers on your machine, GPU will not be used.
2026-04-23 21:43:18.755978: I tensorflow/core/platform/cpu_feature_guard.cc:182] Thi
s TensorFlow binary is optimized to use available CPU instructions in performance-cr
itical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operati
ons, rebuild TensorFlow with the appropriate compiler flags.
2026-04-23 21:43:20.470727: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38]
TF-TRT Warning: Could not find TensorRT
TensorFlow version: 2.14.1
```

2. Download and Extract the UCI HAR Dataset

The UCI HAR dataset contains:

- **561 numerical features** extracted from smartphone sensor signals
- **6 human activity classes**
- A predefined **training split** and **test split**

This makes it a strong fit for a **fully connected DNN** and for TinyML-oriented compression experiments.

In [2]:
```
dataset_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%
zip_path = Path("uci_har_dataset.zip")
extract_dir = Path(".")
```

```
if not zip_path.exists():
    print("Downloading dataset...")
    urllib.request.urlretrieve(dataset_url, zip_path)

dataset_root = Path("UCI HAR Dataset")
if not dataset_root.exists():
    print("Extracting dataset...")
    with zipfile.ZipFile(zip_path, "r") as zf:
        zf.extractall(extract_dir)

print("Dataset ready at:", dataset_root.resolve())
```

```
Downloading dataset...
Dataset ready at: /mnt/c/Users/redey/Desktop/School/EE 446/Lab3/tinyml_env_setup_pac
kage/tinyml_env_setup_package/Lab3/UCI HAR Dataset
```

3. Load the Data

In [3]:

```
def load_har_data(root_dir="UCI HAR Dataset"):
    root_dir = Path(root_dir)

    # TODO:
    # 1. Load X_train from train/X_train.txt as float32
    # 2. Load y_train from train/y_train.txt as int32 and subtract 1
    # 3. Load X_test from test/X_test.txt as float32
    # 4. Load y_test from test/y_test.txt as int32 and subtract 1

    X_train = np.loadtxt(root_dir / "train" / "X_train.txt", dtype=np.float32)
    y_train = np.loadtxt(root_dir / "train" / "y_train.txt", dtype=np.int32) - 1
    X_test = np.loadtxt(root_dir / "test" / "X_test.txt", dtype=np.float32)
    y_test = np.loadtxt(root_dir / "test" / "y_test.txt", dtype=np.int32) - 1

    return X_train, y_train, X_test, y_test

X_train, y_train, X_test, y_test = load_har_data(dataset_root)

class_names = [
    "WALKING",
    "WALKING_UPSTAIRS",
    "WALKING_DOWNSTAIRS",
    "SITTING",
    "STANDING",
    "LAYING",
]

num_features = X_train.shape[1]
num_classes = len(class_names)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape :", X_test.shape)
print("y_test shape :", y_test.shape)
print("Number of features:", num_features)
print("Number of classes :", num_classes)
```

```
X_train shape: (7352, 561)
y_train shape: (7352,)
X_test shape : (2947, 561)
y_test shape : (2947,)
Number of features: 561
Number of classes : 6
```

4. Quick Inspection

In [4]:
```
label_counts = pd.Series(y_train).value_counts().sort_index()

dataset_summary = pd.DataFrame({
    "Class Index": list(range(num_classes)),
    "Class Name": class_names,
    "Training Samples": label_counts.values,
})

dataset_summary
```

Out[4]:

	Class Index	Class Name	Training Samples
0	0	WALKING	1226
1	1	WALKING_UPSTAIRS	1073
2	2	WALKING_DOWNSTAIRS	986
3	3	SITTING	1286
4	4	STANDING	1374
5	5	LAYING	1407

5. Define the Teacher and Student Models

The **teacher model** is intentionally larger and more expressive.

The **student model** is smaller and is the model we ultimately want to deploy.

In [5]:
```
def build_teacher_model(input_dim, num_classes):
    # TODO:
    # Build a larger teacher DNN suitable for 561 numerical input features.
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(256, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(128, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(64, activation="relu"),
        layers.Dense(num_classes, activation="softmax"),
    ])
    return model

def build_student_model(input_dim, num_classes):
```

```
    # TODO:
    # Build a smaller student DNN.
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dense(32, activation="relu"),
        layers.Dense(num_classes, activation="softmax"),
    ])
    return model

teacher_model = build_teacher_model(num_features, num_classes)
student_baseline_model = build_student_model(num_features, num_classes)

teacher_model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense (Dense)               (None, 256)               143872

 dropout (Dropout)           (None, 256)               0

 dense_1 (Dense)             (None, 128)               32896

 dropout_1 (Dropout)         (None, 128)               0

 dense_2 (Dense)             (None, 64)                8256

 dense_3 (Dense)             (None, 6)                 390

=================================================================
Total params: 185414 (724.27 KB)
Trainable params: 185414 (724.27 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

6. Train the Teacher Model

In [6]:

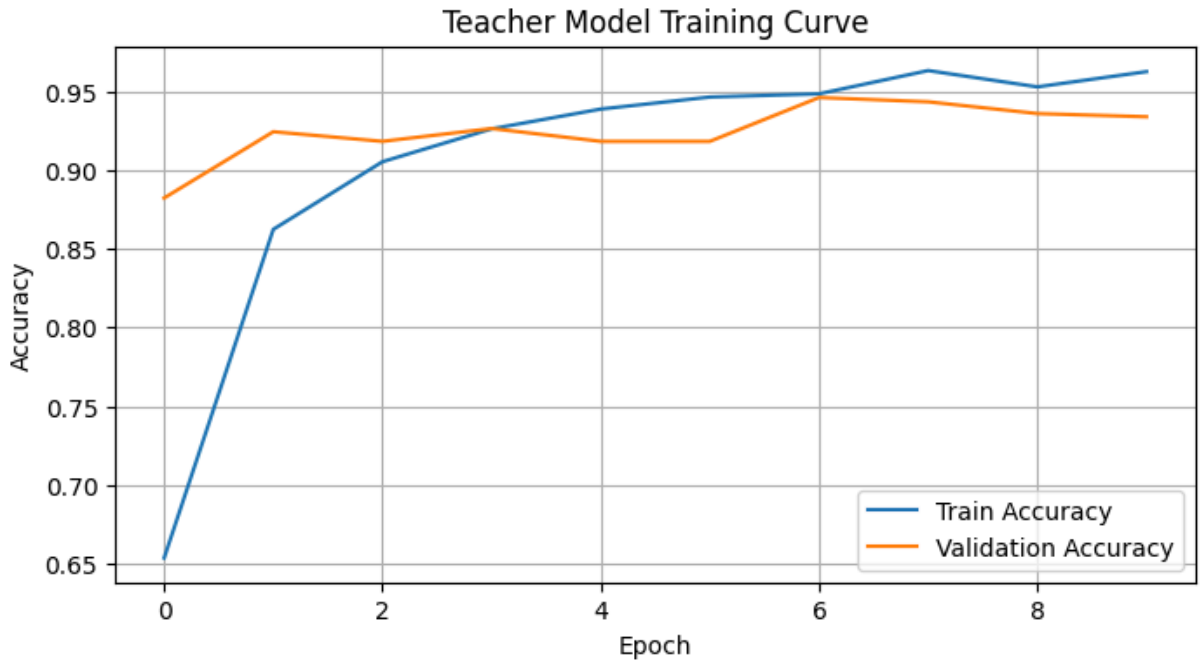
```
teacher_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

teacher_callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True
    )
]

# TODO:
# Train the teacher model on the UCI HAR training split.
```

```
teacher_history = teacher_model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=64,
    callbacks=teacher_callbacks,
    verbose=1
)
```

```
Epoch 1/20
92/92 [==============================] - 1s 6ms/step - loss: 0.7961 - accuracy: 0.65
35 - val_loss: 0.3059 - val_accuracy: 0.8824
Epoch 2/20
92/92 [==============================] - 0s 4ms/step - loss: 0.3312 - accuracy: 0.86
24 - val_loss: 0.1725 - val_accuracy: 0.9245
Epoch 3/20
92/92 [==============================] - 0s 4ms/step - loss: 0.2280 - accuracy: 0.90
55 - val_loss: 0.2164 - val_accuracy: 0.9184
Epoch 4/20
92/92 [==============================] - 0s 4ms/step - loss: 0.1911 - accuracy: 0.92
64 - val_loss: 0.2092 - val_accuracy: 0.9266
Epoch 5/20
92/92 [==============================] - 0s 4ms/step - loss: 0.1472 - accuracy: 0.93
90 - val_loss: 0.2117 - val_accuracy: 0.9184
Epoch 6/20
92/92 [==============================] - 0s 4ms/step - loss: 0.1361 - accuracy: 0.94
66 - val_loss: 0.3040 - val_accuracy: 0.9184
Epoch 7/20
92/92 [==============================] - 0s 4ms/step - loss: 0.1355 - accuracy: 0.94
86 - val_loss: 0.1303 - val_accuracy: 0.9463
Epoch 8/20
92/92 [==============================] - 0s 4ms/step - loss: 0.0956 - accuracy: 0.96
34 - val_loss: 0.1791 - val_accuracy: 0.9436
Epoch 9/20
92/92 [==============================] - 0s 4ms/step - loss: 0.1157 - accuracy: 0.95
31 - val_loss: 0.2061 - val_accuracy: 0.9361
Epoch 10/20
92/92 [==============================] - 0s 5ms/step - loss: 0.1035 - accuracy: 0.96
28 - val_loss: 0.2147 - val_accuracy: 0.9341
```

Teacher Training Curves

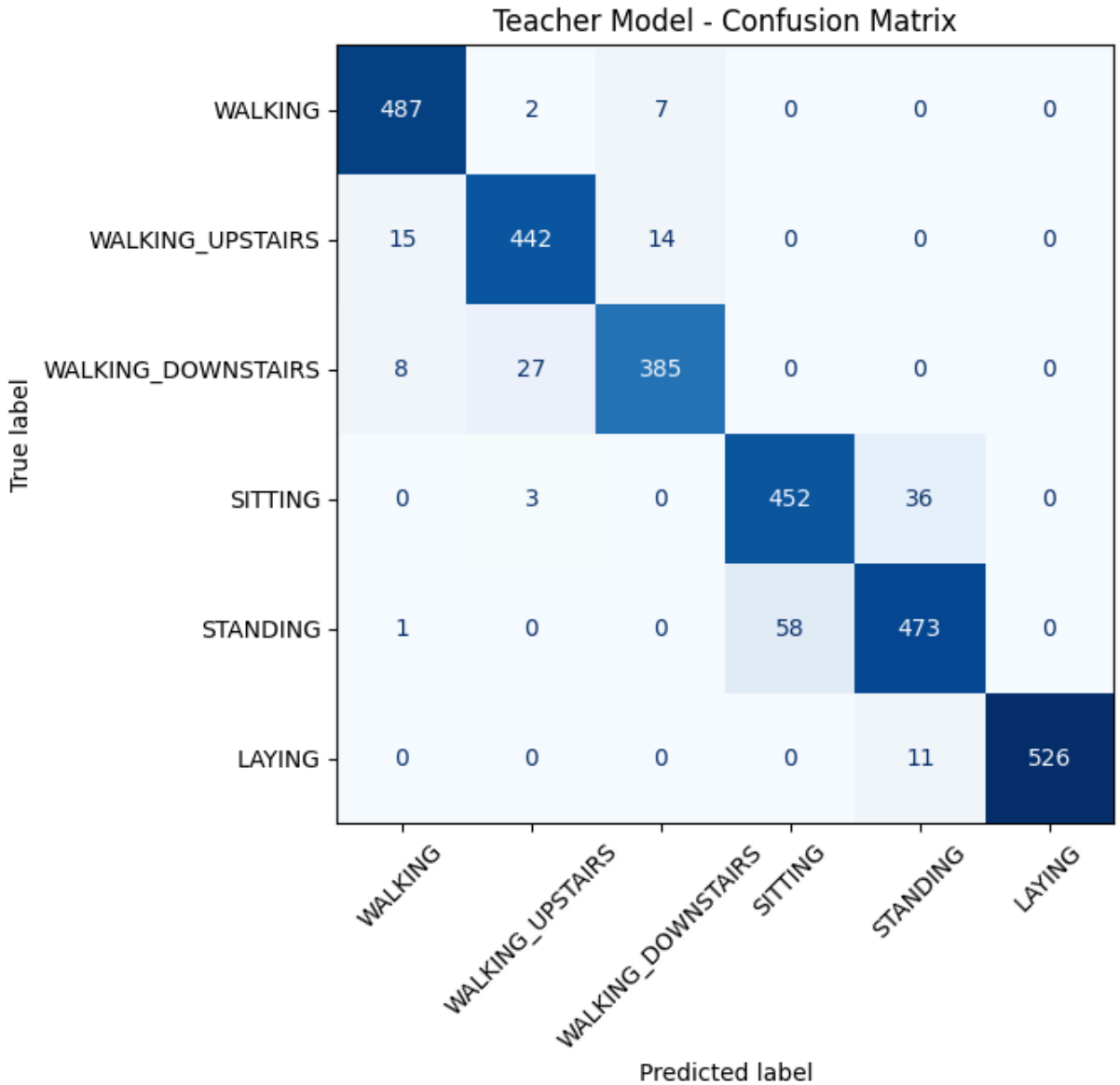
In [7]:
```
teacher_history_df = pd.DataFrame(teacher_history.history)

plt.figure(figsize=(8, 4))
plt.plot(teacher_history_df["accuracy"], label="Train Accuracy")
plt.plot(teacher_history_df["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Teacher Model Training Curve")
plt.legend()
plt.grid(True)
plt.show()
```



7. Evaluate the Teacher Model

In [8]:
```
teacher_probs = teacher_model.predict(X_test, verbose=0)
teacher_preds = np.argmax(teacher_probs, axis=1)
teacher_acc = accuracy_score(y_test, teacher_preds)

print(f"Teacher Test Accuracy: {teacher_acc:.4f}\n")
print(classification_report(y_test, teacher_preds, target_names=class_names, digits

disp = ConfusionMatrixDisplay(
    confusion_matrix=confusion_matrix(y_test, teacher_preds),
    display_labels=class_names
)
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, xticks_rotation=45, cmap="Blues", colorbar=False)
plt.title("Teacher Model - Confusion Matrix")
plt.show()
```

```
Teacher Test Accuracy: 0.9382

                    precision    recall  f1-score   support

           WALKING     0.9530    0.9819    0.9672       496
  WALKING_UPSTAIRS     0.9325    0.9384    0.9354       471
WALKING_DOWNSTAIRS     0.9483    0.9167    0.9322       420
           SITTING     0.8863    0.9206    0.9031       491
          STANDING     0.9096    0.8891    0.8992       532
            LAYING     1.0000    0.9795    0.9897       537

          accuracy                         0.9382      2947
         macro avg     0.9383    0.9377    0.9378      2947
      weighted avg     0.9387    0.9382    0.9383      2947
```



8. Train a Baseline Student Model (Hard Labels Only)

Before applying knowledge distillation, we train the smaller student model in the standard way.

This gives us a fair baseline for comparison.

In [9]:
```
student_baseline_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

student_callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
```

```
        restore_best_weights=True
    )
]

# TODO:
# Train the baseline student using the hard labels only.
student_baseline_history = student_baseline_model.fit(
    #<--- Enter your code here --->#
    X_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=64,
    callbacks=student_callbacks,
    verbose=1
)
```

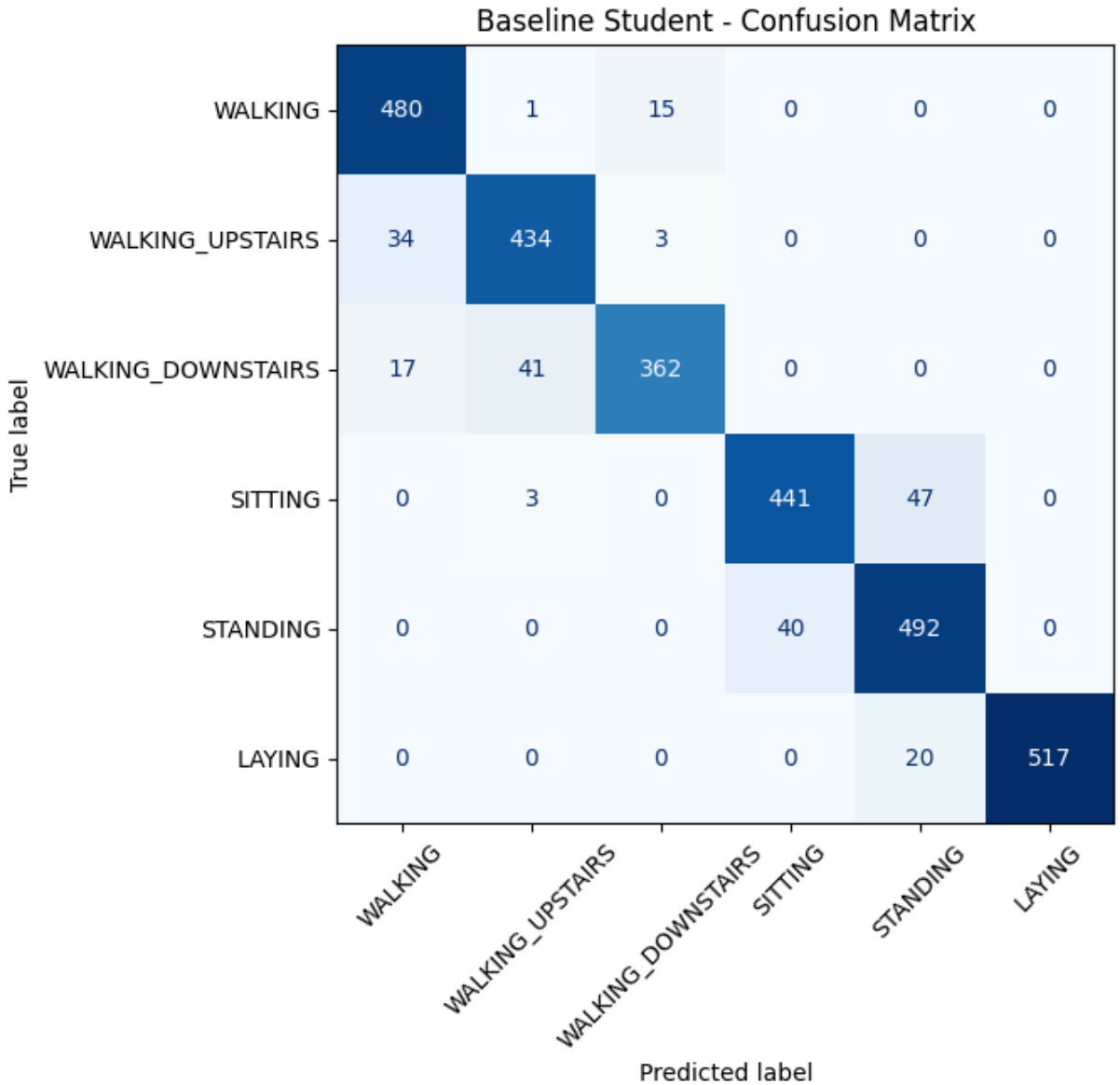
```
Epoch 1/20
92/92 [==============================] - 1s 5ms/step - loss: 0.6947 - accuracy: 0.76
74 - val_loss: 0.3262 - val_accuracy: 0.8906
Epoch 2/20
92/92 [==============================] - 0s 3ms/step - loss: 0.2689 - accuracy: 0.90
78 - val_loss: 0.2485 - val_accuracy: 0.9137
Epoch 3/20
92/92 [==============================] - 0s 3ms/step - loss: 0.1705 - accuracy: 0.94
41 - val_loss: 0.2348 - val_accuracy: 0.9157
Epoch 4/20
92/92 [==============================] - 0s 3ms/step - loss: 0.1358 - accuracy: 0.95
34 - val_loss: 0.1561 - val_accuracy: 0.9320
Epoch 5/20
92/92 [==============================] - 0s 3ms/step - loss: 0.1074 - accuracy: 0.96
21 - val_loss: 0.1495 - val_accuracy: 0.9279
Epoch 6/20
92/92 [==============================] - 0s 3ms/step - loss: 0.0854 - accuracy: 0.97
25 - val_loss: 0.2112 - val_accuracy: 0.9293
Epoch 7/20
92/92 [==============================] - 0s 3ms/step - loss: 0.0778 - accuracy: 0.97
47 - val_loss: 0.1267 - val_accuracy: 0.9436
Epoch 8/20
92/92 [==============================] - 0s 3ms/step - loss: 0.0651 - accuracy: 0.97
93 - val_loss: 0.1407 - val_accuracy: 0.9402
Epoch 9/20
92/92 [==============================] - 0s 3ms/step - loss: 0.0826 - accuracy: 0.96
85 - val_loss: 0.1519 - val_accuracy: 0.9415
Epoch 10/20
92/92 [==============================] - 0s 3ms/step - loss: 0.0623 - accuracy: 0.97
64 - val_loss: 0.1874 - val_accuracy: 0.9341
```

9. Evaluate the Baseline Student Model

In [10]:
```
student_baseline_probs = student_baseline_model.predict(X_test, verbose=0)
student_baseline_preds = np.argmax(student_baseline_probs, axis=1)
student_baseline_acc = accuracy_score(y_test, student_baseline_preds)

print(f"Baseline Student Test Accuracy: {student_baseline_acc:.4f}\n")
```

```
print(classification_report(y_test, student_baseline_preds, target_names=class_name

disp = ConfusionMatrixDisplay(
    confusion_matrix=confusion_matrix(y_test, student_baseline_preds),
    display_labels=class_names
)
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, xticks_rotation=45, cmap="Blues", colorbar=False)
plt.title("Baseline Student - Confusion Matrix")
plt.show()
```

```
Baseline Student Test Accuracy: 0.9250

                    precision    recall  f1-score   support

           WALKING     0.9040    0.9677    0.9348       496
  WALKING_UPSTAIRS     0.9061    0.9214    0.9137       471
WALKING_DOWNSTAIRS     0.9526    0.8619    0.9050       420
           SITTING     0.9168    0.8982    0.9074       491
          STANDING     0.8801    0.9248    0.9019       532
            LAYING     1.0000    0.9628    0.9810       537

          accuracy                         0.9250      2947
         macro avg     0.9266    0.9228    0.9240      2947
      weighted avg     0.9266    0.9250    0.9251      2947
```



Part I: Knowledge Distillation

10. Distillation Utilities

The distilled student is trained to optimize:

- a **hard-label loss** using the true class labels
- a **soft-label loss** using the teacher's softened probability distribution

In [12]:
```
class Distiller(keras.Model):
    def __init__(self, student, teacher):
        super().__init__()
        self.teacher = teacher
        self.student = student

    def compile(
```

```
        self,
        optimizer,
        metrics,
        student_loss_fn,
        distillation_loss_fn,
        alpha=0.3,
        temperature=4.0,
    ):
        super().compile(optimizer=optimizer, metrics=metrics)
        self.student_loss_fn = student_loss_fn
        self.distillation_loss_fn = distillation_loss_fn
        self.alpha = alpha
        self.temperature = temperature

    def train_step(self, data):
        x, y = data

        # TODO:
        # 1. Obtain teacher predictions with training=False
        # 2. Compute student predictions inside GradientTape
        # 3. Compute student_loss using the hard labels
        # 4. Compute distillation_loss using softened teacher/student outputs
        # 5. Combine the two losses using alpha
        teacher_predictions = self.teacher(x, training=False) #<--- Enter your code

        with tf.GradientTape() as tape:
            student_predictions = self.student(x, training=True)
            student_loss = self.student_loss_fn(y, student_predictions)
            distillation_loss = self.distillation_loss_fn(
                tf.nn.softmax(teacher_predictions / self.temperature, axis=1),
                tf.nn.softmax(student_predictions / self.temperature, axis=1),
            ) * (self.temperature ** 2)
            loss = self.alpha * student_loss + (1 - self.alpha) * distillation_loss

        trainable_vars = self.student.trainable_variables
        gradients = tape.gradient(loss, trainable_vars)
        self.optimizer.apply_gradients(zip(gradients, trainable_vars))

        self.compiled_metrics.update_state(y, student_predictions)

        results = {m.name: m.result() for m in self.metrics}
        results.update({
            "student_loss": student_loss,
            "distillation_loss": distillation_loss,
        })
        return results

    def test_step(self, data):
        x, y = data
        y_prediction = self.student(x, training=False)

        student_loss = self.student_loss_fn(y, y_prediction)
        self.compiled_metrics.update_state(y, y_prediction)

        results = {m.name: m.result() for m in self.metrics}
```

```
        results.update({"student_loss": student_loss})
        return results
```

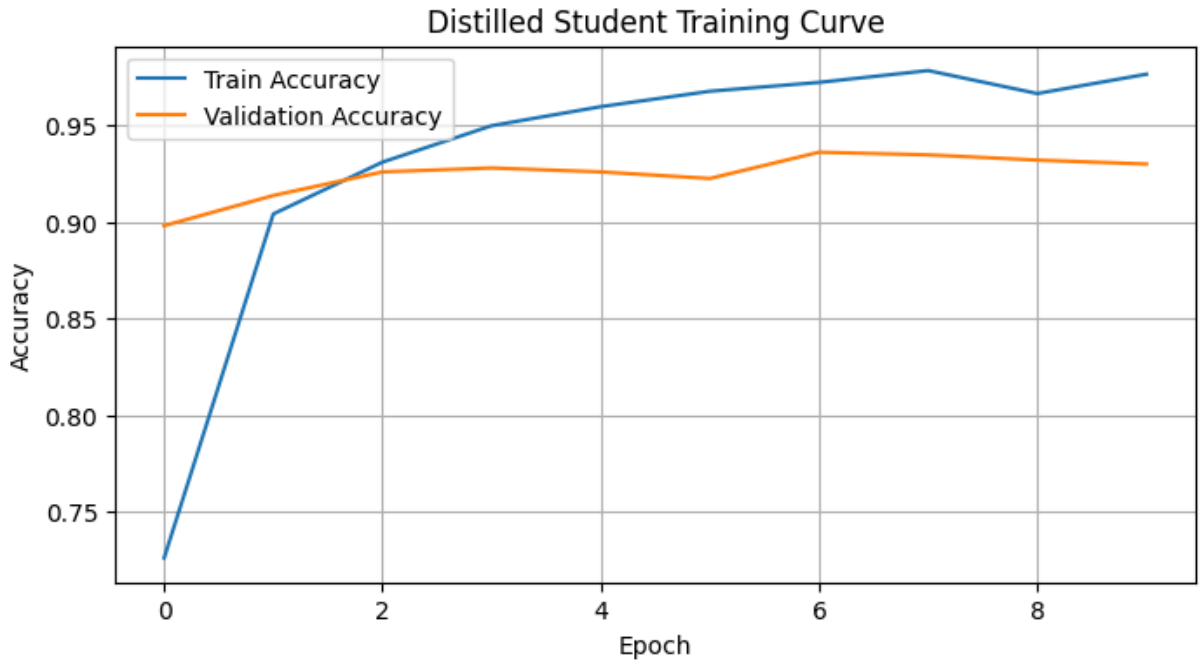
11. Train the Distilled Student

In [13]:
```
distilled_student = build_student_model(num_features, num_classes)

distiller = Distiller(student=distilled_student, teacher=teacher_model)
distiller.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    metrics=[keras.metrics.SparseCategoricalAccuracy(name="accuracy")],
    student_loss_fn=keras.losses.SparseCategoricalCrossentropy(),
    distillation_loss_fn=keras.losses.KLDivergence(),
    alpha=0.3,
    temperature=4.0,
)

distillation_callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True
    )
]

distillation_history = distiller.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=64,
    callbacks=distillation_callbacks,
    verbose=1
)
```

```
Epoch 1/20
92/92 [==============================] - 1s 3ms/step - accuracy: 0.7259 - student_lo
ss: 0.7625 - distillation_loss: 0.0330 - val_accuracy: 0.8980 - val_student_loss: 0.
1741
Epoch 2/20
92/92 [==============================] - 0s 3ms/step - accuracy: 0.9041 - student_lo
ss: 0.2817 - distillation_loss: 0.0099 - val_accuracy: 0.9137 - val_student_loss: 0.
0206
Epoch 3/20
92/92 [==============================] - 0s 2ms/step - accuracy: 0.9310 - student_lo
ss: 0.1876 - distillation_loss: 0.0053 - val_accuracy: 0.9259 - val_student_loss: 0.
0111
Epoch 4/20
92/92 [==============================] - 0s 3ms/step - accuracy: 0.9498 - student_lo
ss: 0.1387 - distillation_loss: 0.0030 - val_accuracy: 0.9279 - val_student_loss: 0.
0069
Epoch 5/20
92/92 [==============================] - 0s 2ms/step - accuracy: 0.9597 - student_lo
ss: 0.1164 - distillation_loss: 0.0023 - val_accuracy: 0.9259 - val_student_loss: 0.
0026
Epoch 6/20
92/92 [==============================] - 0s 2ms/step - accuracy: 0.9677 - student_lo
ss: 0.0933 - distillation_loss: 0.0014 - val_accuracy: 0.9225 - val_student_loss: 0.
0089
Epoch 7/20
92/92 [==============================] - 0s 3ms/step - accuracy: 0.9723 - student_lo
ss: 0.0818 - distillation_loss: 0.0015 - val_accuracy: 0.9361 - val_student_loss: 0.
0017
Epoch 8/20
92/92 [==============================] - 0s 3ms/step - accuracy: 0.9784 - student_lo
ss: 0.0675 - distillation_loss: 9.1478e-04 - val_accuracy: 0.9347 - val_student_los
s: 0.0015
Epoch 9/20
92/92 [==============================] - 0s 3ms/step - accuracy: 0.9665 - student_lo
ss: 0.0857 - distillation_loss: 0.0022 - val_accuracy: 0.9320 - val_student_loss: 0.
0010
Epoch 10/20
92/92 [==============================] - 0s 3ms/step - accuracy: 0.9765 - student_lo
ss: 0.0632 - distillation_loss: 9.8627e-04 - val_accuracy: 0.9300 - val_student_los
s: 0.0021
```

Distillation Training Curves

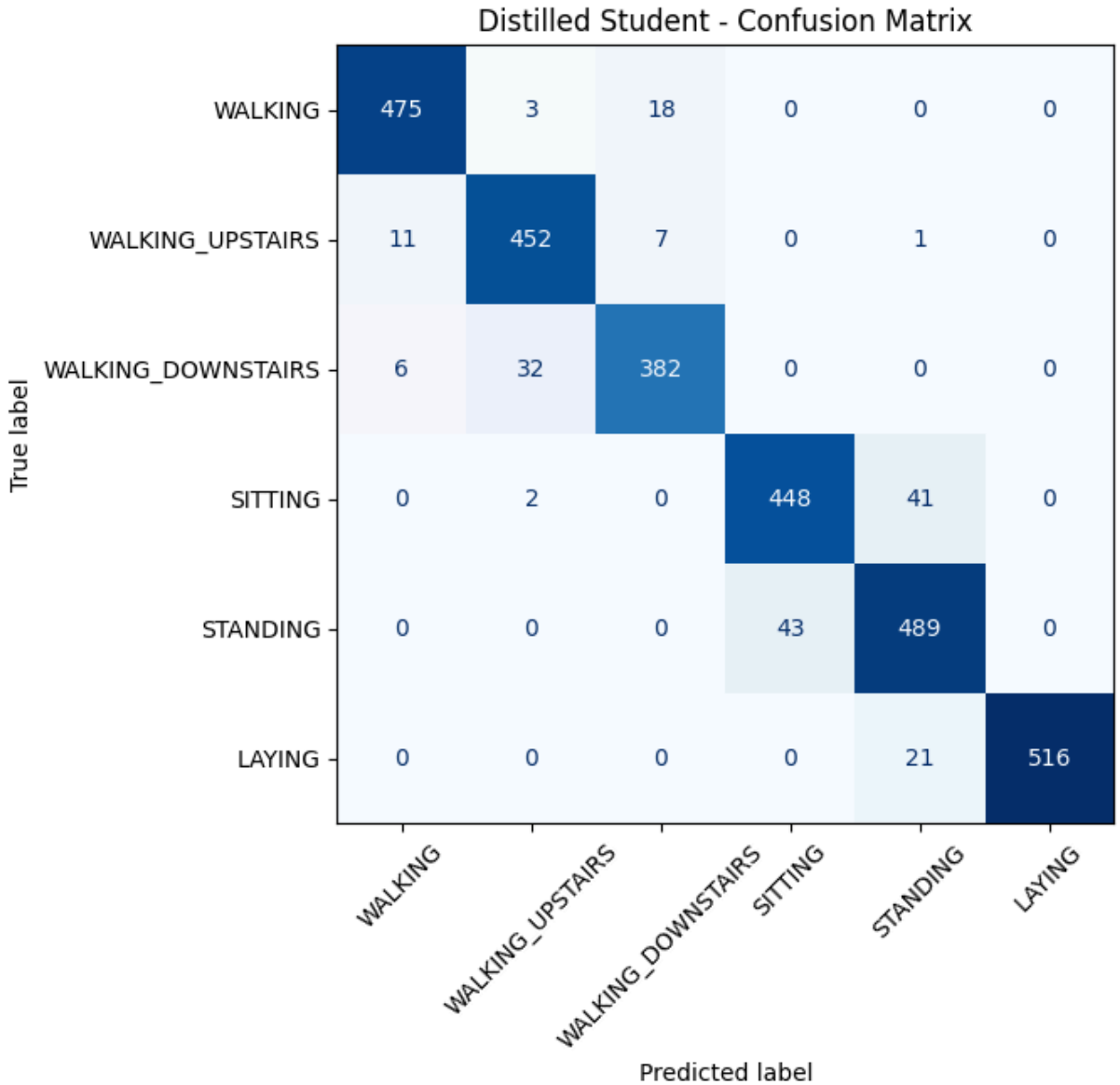
In [14]:
```
distillation_history_df = pd.DataFrame(distillation_history.history)

plt.figure(figsize=(8, 4))
plt.plot(distillation_history_df["accuracy"], label="Train Accuracy")
plt.plot(distillation_history_df["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Distilled Student Training Curve")
plt.legend()
plt.grid(True)
plt.show()
```



12. Evaluate the Distilled Student

In [15]:
```
distilled_probs = distilled_student.predict(X_test, verbose=0)
distilled_preds = np.argmax(distilled_probs, axis=1)
distilled_acc = accuracy_score(y_test, distilled_preds)

print(f"Distilled Student Test Accuracy: {distilled_acc:.4f}\n")
print(classification_report(y_test, distilled_preds, target_names=class_names, digi

disp = ConfusionMatrixDisplay(
    confusion_matrix=confusion_matrix(y_test, distilled_preds),
    display_labels=class_names
)
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, xticks_rotation=45, cmap="Blues", colorbar=False)
plt.title("Distilled Student - Confusion Matrix")
plt.show()
```

```
Distilled Student Test Accuracy: 0.9372

                    precision    recall  f1-score   support

           WALKING     0.9654    0.9577    0.9615       496
  WALKING_UPSTAIRS     0.9243    0.9597    0.9417       471
WALKING_DOWNSTAIRS     0.9386    0.9095    0.9238       420
           SITTING     0.9124    0.9124    0.9124       491
          STANDING     0.8859    0.9192    0.9022       532
            LAYING     1.0000    0.9609    0.9801       537

          accuracy                         0.9372      2947
         macro avg     0.9378    0.9366    0.9370      2947
      weighted avg     0.9381    0.9372    0.9375      2947
```



13. Part I Comparison: Teacher vs Student vs Distilled Student

In [16]:
```
part1_results = pd.DataFrame([
    ["Teacher", teacher_acc],
    ["Baseline Student", student_baseline_acc],
    ["Distilled Student", distilled_acc],
], columns=["Model", "Test Accuracy"])

part1_results
```

Out[16]:

	Model	Test Accuracy
0	Teacher	0.938242
1	Baseline Student	0.925008
2	Distilled Student	0.937224

Part II: Pruning and Quantization of the Distilled Student

14. TensorFlow Lite Utilities

In [17]:

```
def save_binary_model(model_content, filename):
    with open(filename, "wb") as f:
        f.write(model_content)
    return os.path.getsize(filename) / 1024.0  # KB

def evaluate_tflite_model(tflite_model, X, y_true):
    interpreter = tf.lite.Interpreter(model_content=tflite_model)
    interpreter.allocate_tensors()

    input_details = interpreter.get_input_details()[0]
    output_details = interpreter.get_output_details()[0]

    input_scale, input_zero_point = input_details["quantization"]
    output_scale, output_zero_point = output_details["quantization"]

    y_pred = []

    for i in range(len(X)):
        x = X[i:i+1].astype(np.float32)

        if input_details["dtype"] == np.int8:
            x = np.round(x / input_scale + input_zero_point).astype(np.int8)
        elif input_details["dtype"] == np.uint8:
            x = np.round(x / input_scale + input_zero_point).astype(np.uint8)
        else:
            x = x.astype(input_details["dtype"])

        interpreter.set_tensor(input_details["index"], x)
        interpreter.invoke()

        output = interpreter.get_tensor(output_details["index"])

        if output_details["dtype"] == np.int8:
            output = (output.astype(np.float32) - output_zero_point) * output_scale
        elif output_details["dtype"] == np.uint8:
            output = (output.astype(np.float32) - output_zero_point) * output_scale

        pred = int(np.argmax(output, axis=1)[0])
```

```
        y_pred.append(pred)

    acc = accuracy_score(y_true, y_pred)
    return acc, np.array(y_pred)

def convert_to_tflite_fp32(model):
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    return converter.convert()

def representative_data_gen():
    for i in range(min(200, len(X_train))):
        yield [X_train[i:i+1]]
```

15. Convert the Distilled Student to TensorFlow Lite

In [18]:
```
# Convert the distilled student to a baseline FP32 TensorFlow Lite model.
distilled_fp32_tflite = convert_to_tflite_fp32(distilled_student)
distilled_fp32_size_kb = save_binary_model(distilled_fp32_tflite, "distilled_studen
distilled_fp32_acc, distilled_fp32_preds = evaluate_tflite_model(distilled_fp32_tfl

print(f"Distilled Student FP32 TFLite Accuracy: {distilled_fp32_acc:.4f}")
print(f"Distilled Student FP32 TFLite Size (KB): {distilled_fp32_size_kb:.2f}")
```

```
INFO:tensorflow:Assets written to: /tmp/tmp97jmyjcr/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmp97jmyjcr/assets
2026-04-23 21:45:37.600459: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:45:37.600524: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
```

```
Distilled Student FP32 TFLite Accuracy: 0.9372
Distilled Student FP32 TFLite Size (KB): 151.48
```

```
2026-04-23 21:45:37.601552: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmp97jmyjcr
2026-04-23 21:45:37.602044: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:45:37.602062: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmp97jmyjcr
2026-04-23 21:45:37.603794: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.
cc:382] MLIR V1 optimization pass is not enabled
2026-04-23 21:45:37.604165: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:45:37.624847: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmp97jmyjcr
2026-04-23 21:45:37.630893: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 29313 microseconds.
2026-04-23 21:45:37.669768: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_ut
il.cc:269] disabling MLIR crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIREC
TORY` to enable.
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
```

16. Apply Magnitude-Based Pruning to the Distilled Student

In [19]:
```
pruning_epochs = 10
batch_size = 64
steps_per_epoch = math.ceil((0.8 * len(X_train)) / batch_size)

pruning_params = {
    "pruning_schedule": PolynomialDecay(
        initial_sparsity=0.20,
        final_sparsity=0.85,
        begin_step=0,
        end_step=steps_per_epoch * pruning_epochs,
    )
}

# TODO:
# 1. Clone the distilled student model.
# 2. Copy the distilled student weights into the cloned model.
# 3. Wrap the cloned model using prune_low_magnitude with pruning_params.
student_for_pruning = keras.models.clone_model(distilled_student)
student_for_pruning.set_weights(distilled_student.get_weights())

pruned_distilled_model = prune_low_magnitude(student_for_pruning, **pruning_params)

pruned_distilled_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

pruning_callbacks = [
    UpdatePruningStep(),
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=3,
        restore_best_weights=True
    )
]

# TODO:
# Fine-tune the pruned distilled model using:
# - X_train and y_train
# - validation_split=0.2
# - epochs=pruning_epochs
# - batch_size=batch_size
# - callbacks=pruning_callbacks
# - verbose=1
pruning_history = pruned_distilled_model.fit(
    #<--- Enter your code here --->#
    X_train,
    y_train,
    validation_split=0.2,
    epochs=pruning_epochs,
    batch_size=batch_size,
    callbacks=pruning_callbacks,
    verbose=1
)
```

```
Epoch 1/10
92/92 [==============================] - 2s 4ms/step - loss: 0.0656 - accuracy: 0.98
06 - val_loss: 0.1553 - val_accuracy: 0.9341
Epoch 2/10
92/92 [==============================] - 0s 3ms/step - loss: 0.0776 - accuracy: 0.97
47 - val_loss: 0.1453 - val_accuracy: 0.9375
Epoch 3/10
92/92 [==============================] - 0s 2ms/step - loss: 0.0979 - accuracy: 0.97
26 - val_loss: 0.1440 - val_accuracy: 0.9402
Epoch 4/10
92/92 [==============================] - 0s 2ms/step - loss: 0.1808 - accuracy: 0.94
24 - val_loss: 0.1751 - val_accuracy: 0.9449
Epoch 5/10
92/92 [==============================] - 0s 2ms/step - loss: 0.2946 - accuracy: 0.90
24 - val_loss: 0.2367 - val_accuracy: 0.9497
Epoch 6/10
92/92 [==============================] - 0s 3ms/step - loss: 1.3019 - accuracy: 0.69
07 - val_loss: 1.4438 - val_accuracy: 0.6077
Epoch 7/10
92/92 [==============================] - 0s 2ms/step - loss: 1.1330 - accuracy: 0.66
76 - val_loss: 1.1294 - val_accuracy: 0.6329
Epoch 8/10
92/92 [==============================] - 0s 2ms/step - loss: 0.8448 - accuracy: 0.68
87 - val_loss: 0.7528 - val_accuracy: 0.6540
```

17. Convert the Pruned Distilled Student Before and After Stripping

In [20]:

```
pruned_with_mask_tflite = convert_to_tflite_fp32(pruned_distilled_model)
pruned_with_mask_size_kb = save_binary_model(pruned_with_mask_tflite, "pruned_disti
pruned_with_mask_acc, pruned_with_mask_preds = evaluate_tflite_model(pruned_with_ma

stripped_pruned_model = strip_pruning(pruned_distilled_model)

converter = tf.lite.TFLiteConverter.from_keras_model(stripped_pruned_model)
converter.optimizations = [tf.lite.Optimize.EXPERIMENTAL_SPARSITY]
stripped_sparse_tflite = converter.convert()

stripped_sparse_size_kb = save_binary_model(stripped_sparse_tflite, "distilled_stri
stripped_sparse_acc, stripped_sparse_preds = evaluate_tflite_model(stripped_sparse_

print(f"Pruned distilled model with mask accuracy: {pruned_with_mask_acc:.4f}")
print(f"Pruned distilled model with mask size (KB): {pruned_with_mask_size_kb:.2f}"
print(f"Stripped sparse distilled model accuracy: {stripped_sparse_acc:.4f}")
print(f"Stripped sparse distilled model size (KB): {stripped_sparse_size_kb:.2f}")
```

```
INFO:tensorflow:Assets written to: /tmp/tmpwfdsure3/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmpwfdsure3/assets
2026-04-23 21:45:54.066190: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:45:54.066233: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
2026-04-23 21:45:54.066487: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmpwfdsure3
2026-04-23 21:45:54.069497: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:45:54.069520: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmpwfdsure3
2026-04-23 21:45:54.078919: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:45:54.120952: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmpwfdsure3
2026-04-23 21:45:54.135839: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 69353 microseconds.
INFO:tensorflow:Assets written to: /tmp/tmpmcci6xzm/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmpmcci6xzm/assets
Pruned distilled model with mask accuracy: 0.9226
Pruned distilled model with mask size (KB): 304.25
Stripped sparse distilled model accuracy: 0.9226
Stripped sparse distilled model size (KB): 62.87
```

```
2026-04-23 21:45:54.610201: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:45:54.610244: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
2026-04-23 21:45:54.610374: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmpmcci6xzm
2026-04-23 21:45:54.610826: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:45:54.610835: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmpmcci6xzm
2026-04-23 21:45:54.611736: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:45:54.625568: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmpmcci6xzm
2026-04-23 21:45:54.629776: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 19401 microseconds.
```

18. Apply Full Integer Quantization to the Stripped Sparse Distilled Student

In [21]:

```
# TODO:
# Configure the converter for full integer quantization of the stripped sparse mode
converter = tf.lite.TFLiteConverter.from_keras_model(stripped_pruned_model)
converter.optimizations = [tf.lite.Optimize.DEFAULT, tf.lite.Optimize.EXPERIMENTAL_
converter.representative_dataset = representative_data_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

stripped_sparse_int8_tflite = converter.convert()
```

```
stripped_sparse_int8_size_kb = save_binary_model(
    stripped_sparse_int8_tflite,
    "distilled_stripped_sparse_int8.tflite"
)
stripped_sparse_int8_acc, stripped_sparse_int8_preds = evaluate_tflite_model(
    stripped_sparse_int8_tflite,
    X_test,
    y_test
)

print(f"Stripped Sparse + INT8 Accuracy: {stripped_sparse_int8_acc:.4f}")
print(f"Stripped Sparse + INT8 Size (KB): {stripped_sparse_int8_size_kb:.2f}")
```

```
INFO:tensorflow:Assets written to: /tmp/tmp5ar8e6g0/assets
```

```
INFO:tensorflow:Assets written to: /tmp/tmp5ar8e6g0/assets
/home/redey/ai/projects/tinyml-arduino/lib/python3.11/site-packages/tensorflow/lite/
python/convert.py:947: UserWarning: Statistics for quantized inputs were expected, b
ut not specified; continuing anyway.
  warnings.warn(
2026-04-23 21:46:00.338885: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:378] Ignored output_format.
2026-04-23 21:46:00.338938: W tensorflow/compiler/mlir/lite/python/tf_tfl_flatbuffer
_helpers.cc:381] Ignored drop_control_dependency.
2026-04-23 21:46:00.339098: I tensorflow/cc/saved_model/reader.cc:83] Reading SavedM
odel from: /tmp/tmp5ar8e6g0
2026-04-23 21:46:00.339582: I tensorflow/cc/saved_model/reader.cc:51] Reading meta g
raph with tags { serve }
2026-04-23 21:46:00.339594: I tensorflow/cc/saved_model/reader.cc:146] Reading Saved
Model debug info (if present) from: /tmp/tmp5ar8e6g0
2026-04-23 21:46:00.340625: I tensorflow/cc/saved_model/loader.cc:233] Restoring Sav
edModel bundle.
2026-04-23 21:46:00.351583: I tensorflow/cc/saved_model/loader.cc:217] Running initi
alization op on SavedModel bundle at path: /tmp/tmp5ar8e6g0
2026-04-23 21:46:00.355929: I tensorflow/cc/saved_model/loader.cc:316] SavedModel lo
ad for tags { serve }; Status: success: OK. Took 16830 microseconds.
fully_quantize: 0, inference_type: 6, input_inference_type: INT8, output_inference_t
ype: INT8
Stripped Sparse + INT8 Accuracy: 0.9233
Stripped Sparse + INT8 Size (KB): 33.48
```

19. Part II Comparison: Distillation, Pruning, and Quantization

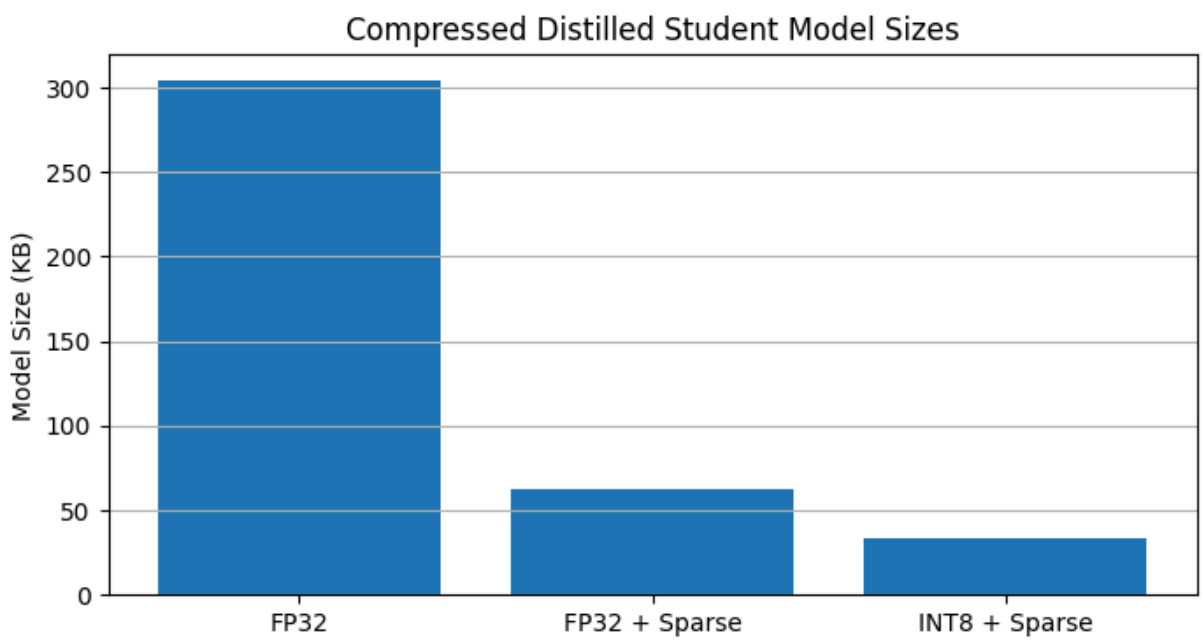
In [22]:
```
part2_results = pd.DataFrame([
    ["Distilled Student TFLite", "FP32", distilled_fp32_acc, distilled_fp32_size_kb
    ["Pruned Distilled TFLite (with mask)", "FP32", pruned_with_mask_acc, pruned_wi
    ["Stripped Sparse Distilled TFLite", "FP32 + Sparse", stripped_sparse_acc, stri
    ["Stripped Sparse Distilled TFLite", "INT8 + Sparse", stripped_sparse_int8_acc,
], columns=["Model", "Format", "Test Accuracy", "Model Size (KB)"])

part2_results
```

Out[22]:

	Model	Format	Test Accuracy	Model Size (KB)
0	Distilled Student TFLite	FP32	0.937224	151.476562
1	Pruned Distilled TFLite (with mask)	FP32	0.922633	304.253906
2	Stripped Sparse Distilled TFLite	FP32 + Sparse	0.922633	62.867188
3	Stripped Sparse Distilled TFLite	INT8 + Sparse	0.923312	33.476562

In [23]:

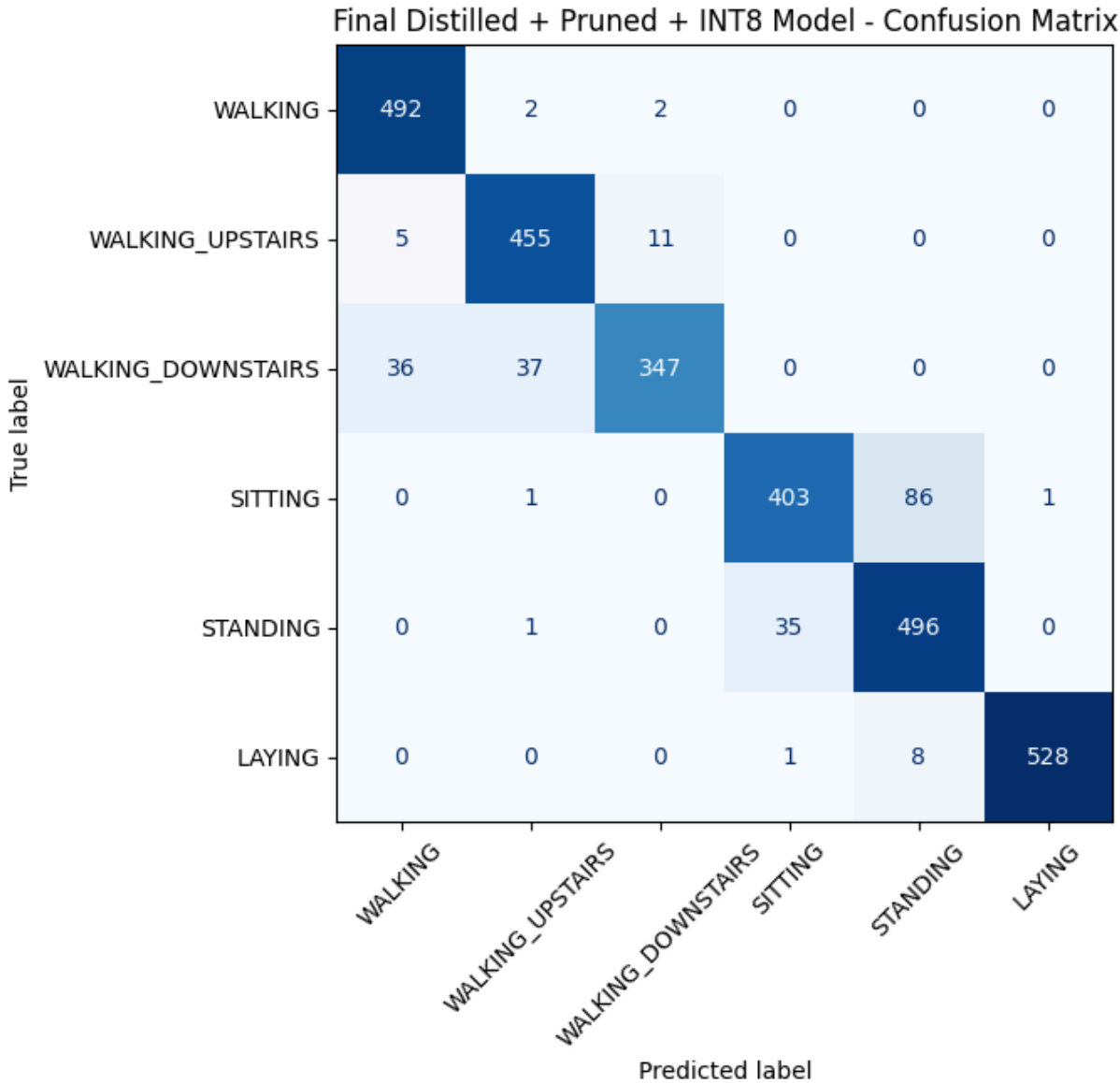
```
plt.figure(figsize=(8, 4))
plt.bar(part2_results["Format"], part2_results["Model Size (KB)"])
plt.ylabel("Model Size (KB)")
plt.title("Compressed Distilled Student Model Sizes")
plt.grid(axis="y")
plt.show()
```



Confusion Matrix for the Final Sparse INT8 Distilled Student

In [24]:

```
disp = ConfusionMatrixDisplay(
    confusion_matrix=confusion_matrix(y_test, stripped_sparse_int8_preds),
    display_labels=class_names
)
fig, ax = plt.subplots(figsize=(8, 6))
disp.plot(ax=ax, xticks_rotation=45, cmap="Blues", colorbar=False)
plt.title("Final Distilled + Pruned + INT8 Model - Confusion Matrix")
plt.show()
```



20. Summary Questions

Answer the following in your lab report:

1. How did the **baseline student** compare with the **distilled student**?
2. Did **knowledge distillation** help the smaller model retain performance?
3. What happened to the model size after **pruning** and after **INT8 quantization**?
4. Which model would you choose for **Arduino deployment**, and why?
5. Why is the final **sparse INT8 model** a good TinyML deployment candidate?

Summary Answers

1. The distilled student performed better than the baseline student, achieving higher accuracy despite having the same smaller architecture.

2. Yes, knowledge distillation helped the smaller model retain performance by transferring knowledge from the larger teacher model to the smaller student model.
3. The model size decreased significantly after pruning and decreased even further after INT8 quantization.
4. I would choose the distilled sparse INT8 model for Arduino deployment because it provides a strong balance between model size, inference speed, and accuracy.
5. The final sparse INT8 model is a good TinyML deployment candidate because it combines model compression techniques that reduce memory usage and computation requirements while maintaining acceptable accuracy.

21. Submission Requirements

Submit the following:

1. Your completed notebook
2. Screenshots of the most important results:
   - teacher accuracy
   - baseline student accuracy
   - distilled student accuracy
   - final sparse INT8 model accuracy and size
3. The exported TensorFlow Lite model:
   - `distilled_stripped_sparse_int8.tflite`
4. Short answers to the summary questions